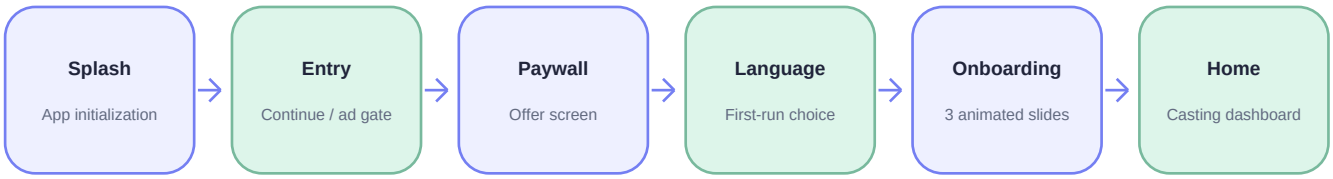
CASTFLOW

Updated Flutter Implementation Plan

Product flow • Local-first architecture • Ads & monetization • Paywall • TV remote • UI/UX • QA



Planning baseline: Existing CastFlow plan dated 1 November 2026, updated with the new requirements in this discussion.

Core constraint: No company backend, no API server and no cloud database. App preferences, ad counters, history and device metadata remain on-device. TV discovery and casting communicate over the local network.

Prepared as an implementation and QA handoff document

1. Product direction and scope

CastFlow remains a Flutter screen-mirroring and media-casting app. The first release is Android-focused, while the codebase must be structured for a later iOS release without redesigning the shared product logic.

Area	Decision	Implementation note
Backend	None	No custom server, REST API, user account or cloud database. Do not introduce Firebase or a hosted service for core functionality.
Local persistence	On-device only	Use shared_preferences for small settings and Hive CE (or an equivalent local store) for history, saved devices and structured local data.
Platforms	Android first; iOS-ready	Keep platform-specific capture, discovery and receiver SDK code behind interfaces and platform channels.
TV communication	Local Wi-Fi	Discovery and media delivery remain on the local network. Explain same-Wi-Fi requirements and router isolation states.
Monetization	Ad SDK + local entitlement state	Ad requests and eligibility are controlled in-app. If subscriptions are sold, use store-native billing; do not assume server-side receipt validation exists.

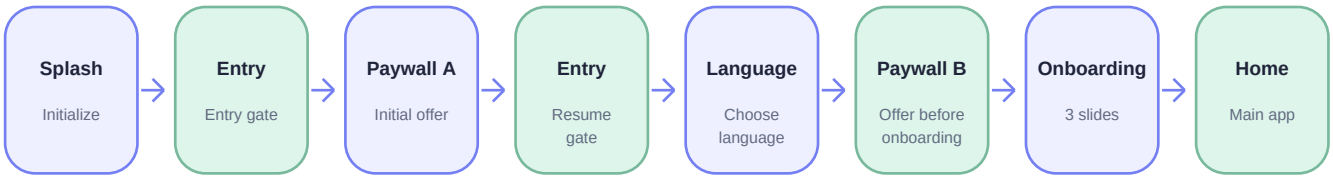
Important feasibility note

A fully offline/local-first product can still use third-party advertising and app-store billing SDKs, but those SDKs communicate with their providers. “No backend” means CastFlow will not operate its own backend. If the requirement is literally no network communication except the TV, ad serving and subscription verification cannot work as normal store services; this must be accepted as a product constraint before development.

2. App navigation and user journeys

Treat “Entry” as the lightweight entry/continue gate screen between Splash and the next destination. Keep it as a single reusable route; do not create duplicate entry screens. The exact sequence requested is captured below.

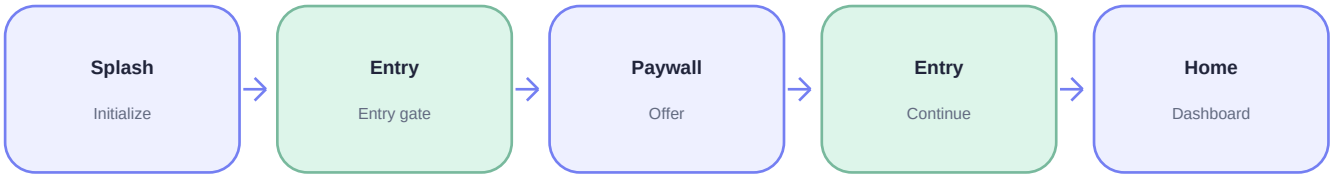
2.1 First-time user



First launch route: Splash → Entry → Paywall A → Entry → Language → Paywall B → 3-slide Onboarding → Home.

Show the language-screen native ad in the language screen itself, and the onboarding native ad in the onboarding surface. Never request these native ads early on a different screen merely to preload them.

2.2 Returning user



Returning route: Splash → Entry → Paywall → Entry → Home. Do not show first-run language selection or the three onboarding slides again unless the user explicitly resets onboarding or the product later adds a re-introduction option.

2.3 Routing and state rules

Persist firstLaunchCompleted, onboardingCompleted, selectedLanguage, paywall eligibility/last presentation, and the minimum routing state locally.

Use a single AppFlowController / router guard to decide the next route. Do not scatter first-run checks across individual widgets.

Prevent duplicate paywall presentation during a single app start. The two first-run paywall positions above are intentional only if product owner confirms both are required; otherwise configure one first-run paywall slot and keep the other disabled.

Back navigation must follow the current flow safely: no accidental app exit from a modal/paywall, no duplicate route stacking, and no returning to a completed first-run step.

Persist completion only after the relevant step succeeds; closing the app mid-onboarding must resume at a predictable safe route.

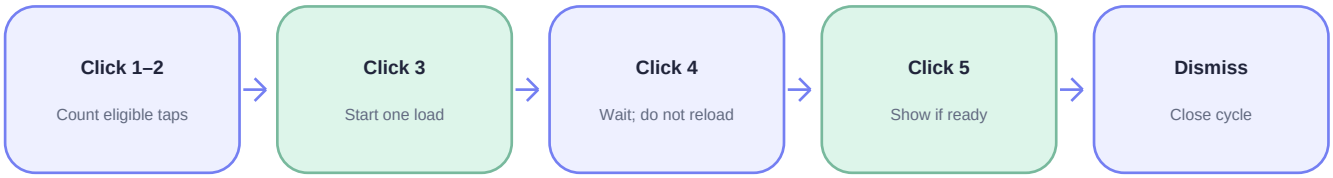
3. Monetization and ad lifecycle

Centralize all ad behavior in an AdManager. Screens may request an eligible placement, but they must not instantiate ad objects or independently decide reload timing. Use explicit placement IDs, state machines and local counters.

3.1 Ad placements

Placement	Where / when	Format / behavior
Language – first run	Inside first-time Language screen	Medium-size native ad. Request/load only when this screen is entered and placement is eligible. Render in reserved layout space after load.
Language – Settings	Inside Settings → Language	Same medium-size native format, independently eligible. Load on this screen, not globally on app launch.
Onboarding	Inside the onboarding flow	Native ad placed within the onboarding layout without interrupting slide controls. Request on the onboarding screen where it is intended to appear.
5-click interstitial	App-wide eligible user actions	Start load at click 3; show at click 5 only if loaded and all frequency/consent rules pass.
App Open	When app returns foreground after being minimized/backgrounded	Request on qualifying resume. Display loading state text “Ad is loading”; show the ready ad on the resume surface, with a back-blocking overlay while the ad is active/loading as specified.

3.2 Five-click interstitial state machine



State / event	Required behavior
Clicks 1–2	Increment eligible click counter. Do not request an interstitial.
Click 3	Start exactly one load request. Mark it as pending. Do not start a second request while this request is pending or ready.
Click 4	Keep the same request alive. Do not reload or reset the click count.
Click 5	If loaded, show the already-loaded ad. If still loading, do not show a stale ad later on an unrelated screen; define a short validity window and discard if the opportunity has passed.
No fill / load error	Do not immediately retry on the next click. End this attempt and wait for the next configured eligibility cycle, with a cooldown to avoid request spam.
Impression + dismiss	Only after the displayed ad is dismissed should the next five-click cycle begin. Do not request another ad at impression or dismiss. The next request is triggered by its own click-3 event.
App background / route change	Do not accidentally show the interstitial over a permission dialog, active casting session transition, paywall, or another full-screen ad. Queue or cancel according to a single documented policy.

Critical clarification: a late-loaded ad must not become eligible merely because its original click counter has advanced. The show decision is tied to the intended fifth-click opportunity. If it misses that opportunity, discard it and wait for the next defined cycle.

3.3 App Open behavior

Trigger only when the app transitions from background/minimized to foreground and the resume rules say an App Open ad is eligible. Do not trigger on ordinary route changes inside the app.

On resume, show the app-level loading surface with the exact copy “Ad is loading”. Keep the app’s underlying screen stable so the user returns to the same context after the ad.

While the App Open loading/overlay surface is active, intercept Android back so it does not pop the underlying screen or exit the app. Restore normal back behavior when the overlay is removed.

If an ad is ready, show that loaded ad on the same resume opportunity. Do not issue a second request just to show it.

Do not reload on impression or dismiss. The next request must occur only at the next qualifying foreground event and only after cooldown/frequency eligibility passes.

If load fails or times out, remove the loading overlay and return the user to the exact screen they were on. Never leave the app blocked indefinitely.

Do not show App Open over the first cold start before the user has reached the app’s usable state, or over another full-screen ad. Maintain a short resume grace period and frequency cap.

3.4 Ad placement and lifecycle rules

Native ads are screen-scoped: request on the actual Language or Onboarding screen where the ad is displayed. Dispose the native ad when the screen is permanently left; retain it only while that screen remains mounted/eligible.

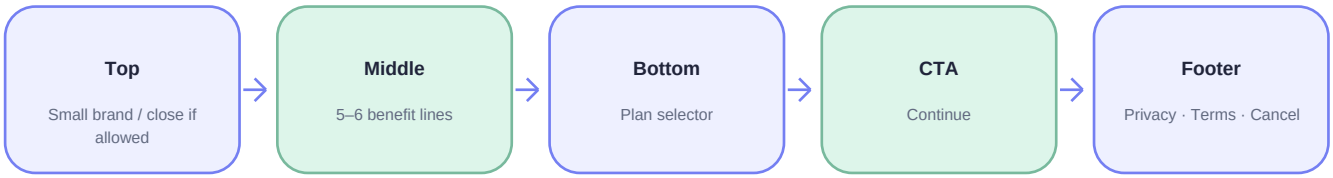
Do not reload an ad on impression or dismiss. AdManager owns one request per placement lifecycle and records request, load, impression, click, dismiss and failure events locally for debugging.

Use remote configuration only if the product later chooses a third-party configuration service. For this no-backend baseline, placement flags and caps are local constants/build configuration and may be changed in a new app release.

Respect consent, privacy disclosures, child-directed settings where applicable, store policies and the ad network’s current SDK requirements. Do not force-show ads when consent or policy disallows them.

4. Paywall specification

The paywall must be deliberately simple, light and conversion-focused. It is not a feature-heavy marketing page.



4.1 Layout

Zone	Requirement
Header	No microphone icon, no decorative mic illustration and no unrelated large hero artwork. Keep brand mark/title restrained. Close/back behavior must be explicitly defined by the product owner and must not silently bypass mandatory legal disclosures.
Benefits	Show five or six concise lines only. Each line should communicate one real benefit (for example: mirror phone screen, cast photos/videos, control playback, browse supported web videos, use TV remote). Avoid unsupported promises.
Plan area	Compact plan selector near the bottom. Show weekly / 7-day plan and any other approved plans with clear price, billing period, renewal terms and trial details if any. Never hardcode example prices such as “80”; load localized price strings from StoreKit / Google Play Billing.
Primary CTA	One clear Continue button. Disable while purchase is processing; show progress and handle cancel, pending, failure and restore outcomes.
Footer	Small links/options: Privacy Policy, Terms & Conditions, and “Cancel anytime” only when factually accurate for the selected subscription. Make links tappable and readable.

4.2 Billing without a custom backend

Use Google Play Billing on Android and StoreKit on iOS through a maintained Flutter in-app-purchase integration. Do not introduce RevenueCat or a custom receipt-validation server if the no-backend requirement is strict. Understand the trade-off: client-only entitlement checks are less resistant to tampering than server-verified receipts. Restore purchases must be available, and entitlement state should be refreshed from the store APIs.

4.3 Paywall QA

• Correct localized price, currency, billing period and renewal wording for each store account region.

• Purchase success, user cancellation, pending purchase, billing unavailable, network interruption and duplicate tap.

• Restore purchases after reinstall and on a second device using the same store account.

• Expired, refunded or revoked subscription state is reflected when the store reports it.

• Privacy Policy and Terms open the correct published documents; no dead links.

• Paywall presentation frequency is consistent with first-time and returning-user routing and cannot stack twice accidentally.

5. Language selection and onboarding

5.1 Language screens

• Use a medium-sized, clean language selection screen—not a full-screen oversized list or tiny dialog. Provide search only if the language list becomes long.

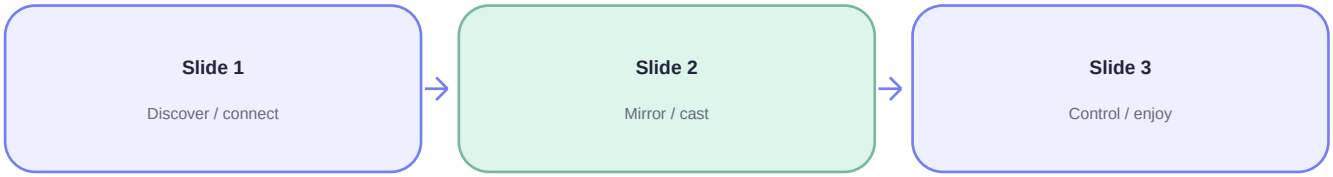
• Use the same visual component in first-time setup and Settings → Language, but keep the two routes' ad eligibility separate.

• Show a medium native ad in the lower portion of the screen with a clear "Ad" label and stable reserved height. Keep language rows, selected state and Continue action unobstructed.

• Apply the chosen locale immediately where practical; persist it locally. Include RTL layout validation for Arabic and Urdu if those languages ship.

• Do not reset onboarding or show first-run paywalls just because the user changes language from Settings.

5.2 Three-slide onboarding



Each slide needs its own motion design and clear value proposition. Suggested creative direction:

Slide	Visual concept	Motion
1 – Connect	Phone scanning a room/network; TV tiles appear and one TV becomes connected.	Soft radar ripple, device cards fade/slide in, connected pulse.
2 – Mirror	Phone screen visually transfers to a large TV display; show a recognizable screen-mirroring scene.	Screen content scales from phone to TV with a smooth light trail; avoid distracting particle overload.
3 – Control	Phone remote UI beside TV playback; highlight play/pause, volume and navigation.	Buttons animate in sequence; TV volume indicator responds to a remote tap.

Keep the three slides swipeable with visible page indicators and Skip / Next / Get Started controls. The onboarding native ad must be integrated without covering artwork, text, dots or navigation. Prefer a clearly separated native placement between content and bottom controls; if there is insufficient space, use a dedicated ad region or a non-ad slide layout rather than compressing the experience.

6. Home, casting and TV remote

6.1 Home layout

• Top area: app greeting/title, connection status chip and Settings shortcut.

• Primary hero: Screen Mirroring / Connect to TV with a strong connected/disconnected state and clear CTA.

• Feature area: Photos, Videos, Web Cast, Music (if in release scope), Apps/Supported services and History shortcuts.

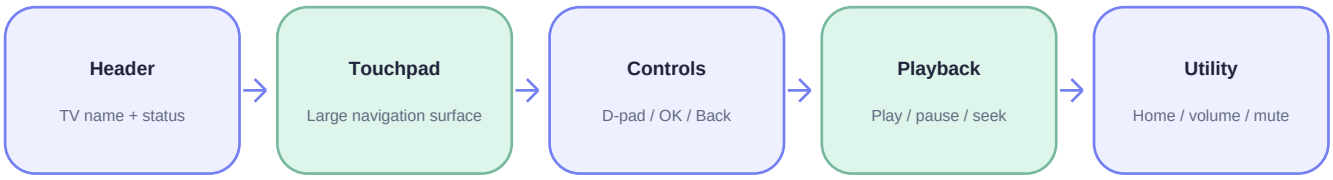
• Add a dedicated TV Remote tile/card on Home. It should open the remote screen when a compatible TV is connected; if not connected, route to device discovery and explain that a TV connection is required.

• Show a compact Now Playing / Mini Cast bar when media is active. It should open the shared Remote / Now Playing screen.

• Keep ad placements out of high-intent controls and avoid placing ads where they could be mistaken for Connect, Mirror or Remote actions.

6.2 Dedicated remote screen

Create a distinctive remote layout inspired by a premium LCD TV remote, not a generic grid of buttons. The remote must be usable one-handed and must adapt to the connected receiver's supported commands.



Remote region	Controls / behavior
Header	Back, connected TV name, connection indicator, optional battery/network status only if genuinely available.
LCD-style display	A compact display panel showing current media title or selected action, playback status and volume level. Do not simulate TV information that the receiver does not expose.
Navigation	Large D-pad with Up / Down / Left / Right and central OK/Select. Use tactile-looking but accessible controls with clear pressed states.
Quick keys	Back, Home, Menu/Input where supported. Hide or disable unsupported commands rather than pretending they work.
Playback	Play/Pause, Previous/Next, seek backward/forward where receiver supports it.
Sound	Volume +/– and Mute where supported. Hardware volume buttons may control TV volume on Android while casting if receiver integration supports it.
Touchpad / gesture area	Optional swipe pad for directional navigation. Include haptic/ripple feedback and a setting to disable haptics.
Disconnect	Visible but secondary Stop casting / Disconnect action with confirmation if it would terminate active playback or mirroring.

Compatibility note: Chromecast, DLNA, Roku, Fire TV and generic Android TV do not expose identical remote-control APIs. Build a capability matrix per receiver. Do not promise universal TV power/input buttons unless the protocol/device actually supports them.

7. Settings and local data

Settings should be organized into clear groups with a premium Pro card at the top. The Pro card should have a small Pro icon and open the simple paywall.

Order	Setting item	Behavior / notes
1	Pro card	Top of Settings. Compact card, Pro icon, short benefit line and Upgrade CTA. Reflect active entitlement if available.
2	Language	Open medium-size language screen with its own native ad placement eligibility.
3	Theme	System / Light / Dark. Apply immediately without restart; keep colors and contrast consistent.
4	Casting preferences	Default quality, auto-connect to last TV, preferred connection behavior and audio preference where supported.
5	Remote preferences	Haptic feedback, remote layout/gesture preference if implemented, and optional button sound toggle.
6	History & storage	Clear casting history, clear browser history/bookmarks if applicable, explain that this does not delete original phone media.
7	Privacy Policy	Open published privacy policy.
8	Terms & Conditions	Open published terms.
9	Feedback / Report a problem	Open email composer with app version, OS version, phone model and receiver type prefilled. User reviews and sends; do not silently transmit data.
10	Rate app	Use official store review flow where available; do not show repeated prompts.
11	Latest version / About	Show installed version/build and open store listing for update. Do not claim an update is available without checking the store.
12	Help & FAQ	Troubleshooting for Wi-Fi, device discovery, connection, mirroring, sound and remote compatibility.

7.1 Local data model additions

Model	Suggested fields	Storage
AppSettings	themeMode, language, defaultQuality, autoConnect, onboardingCompleted, firstLaunchCompleted, hapticsEnabled	SharedPreferences
AdState	placementId, lastRequestAt, lastImpressionAt, clickCycleCount, requestState, cooldownUntil, appOpenLastShownAt	SharedPreferences / local store
EntitlementState	productId, status, expiry if store provides it, lastCheckedAt	Local cache; refresh from store SDK
Device	id, name, ip, port, receiverType, lastSeen, capabilities	Hive CE
HistoryItem	id, mediaTitle, mediaType, deviceName, castAt	Hive CE, bounded list

Never store ad SDK objects, temporary ad handles, live playback streams or sensitive media contents in persistent local storage. Keep only the minimum metadata needed to restore the app experience.

8. Technical architecture and Flutter-to-iOS readiness

Retain the original plan's layered structure: Presentation → State/Use cases → Repositories → Platform/SDK adapters. Use Riverpod for state, go_router for navigation, Freezed models where useful, and platform channels/Pigeon for native functionality.

Layer	Responsibility	Guidance
Presentation	Splash, Entry, Paywall, Language, Onboarding, Home, Remote, Settings	Flutter widgets only; responsive layout and accessible controls.

Layer	Responsibility	Guidance
App flow	First-run / returning routing and route guards	One AppFlowController with persisted milestones and deterministic route decisions.
Monetization	AdManager, placement policies, paywall and billing	Separate AdManager from BillingManager. Ad lifecycle must be testable with fake adapters.
Domain	Cast receiver abstraction, device discovery, playback, remote capabilities	Shared Dart interfaces; avoid receiver-specific logic in widgets.
Data	Local repositories, settings, history and bookmarks	Hive CE + shared_preferences; no cloud database.
Platform	Android capture / services / SDKs; later iOS ReplayKit / Cast / AirPlay	Keep Kotlin and Swift behind Pigeon-generated contracts; no Android-only types in shared Dart models.

Suggested project modules

lib/core/ (theme, router, storage, widgets, platform, utilities) • **lib/features/** (app_flow, monetization, paywall, language, onboarding, home, discovery, connection, mirroring, media_cast, web_cast, remote, history, settings) • **android/** (MediaProjection, foreground service, Cast SDK, discovery) • **ios/** (ReplayKit Broadcast Extension, App Group, Cast/AirPlay adapters when iOS work begins).

iOS readiness checklist

• Keep business rules and routing platform-neutral; do not call Android APIs directly from Dart screens.

• Define interfaces for screen capture, device discovery, receiver control, ad presentation and billing.

• Use platform capability flags so unsupported features are hidden or explained on iOS.

• Keep design tokens, localization ARB files, local models and repositories shared.

• Treat iOS screen mirroring as a separate native implementation using ReplayKit Broadcast Upload Extension; it is not a simple Flutter port.

• Review iOS local-network permissions, Bonjour declarations, multicast entitlement, App Store policies and ad SDK support before committing the iOS schedule.

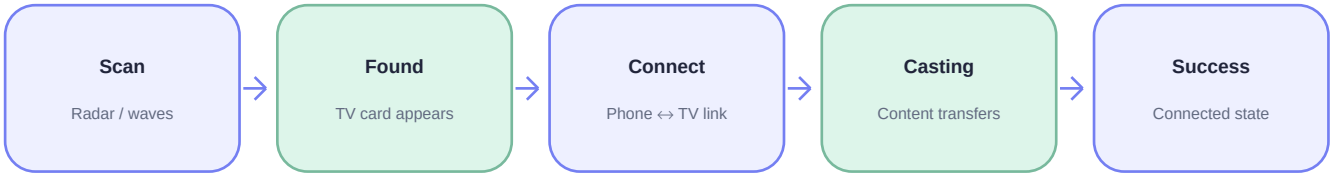
9. Design system, motion and interaction quality

Continue the existing soft visual direction: airy spacing, rounded surfaces, gentle periwinkle gradients, light mode first and a matching dark mode. Keep the current design language consistent rather than introducing a second unrelated theme.

Element	Guideline
Typography	Plus Jakarta Sans or equivalent; clear hierarchy, readable body text, consistent line height.
Cards	20–28 px radius for hero surfaces; avoid excessive nested cards. Use consistent elevation and spacing.
Motion	Use purposeful 200–350 ms transitions; respect reduced-motion preferences and avoid blocking the user.
Loading	Shimmer for lists, ripple for scanning, clear progress for connecting/casting; no endless spinner without recovery.
Feedback	Pressed states, subtle haptics, accessible labels, error copy and retry actions for all important controls.
Animation assets	Use Lottie/Rive or Flutter animations for onboarding and connection states; optimize asset size and frame performance.

9.1 Cash-flow / connection process animations

For the key casting flow, show a continuous, understandable visual story rather than disconnected decorative animations:



• During device discovery, animate a radar/ripple around the phone or TV icon; discovered TV cards should enter smoothly.

• During connection, animate a subtle phone-to-TV connection path or light stream. Show a real state label (Connecting, Connected, Failed).

• During screen mirroring, show a phone preview flowing to a TV frame with a clear active state; do not imply content is being uploaded to a cloud.

• During media casting, animate thumbnail-to-TV transfer, then switch to actual playback controls. Avoid long animations that delay the first frame.

• On success, use a brief connected pulse and then settle into a stable UI. On failure, stop the animation and show a useful retry/help action.

10. QA strategy and acceptance criteria

Area	Must-pass checks
Routing	Fresh install first-user flow; returning-user flow; force close at each step; app upgrade; onboarding reset; no duplicate paywall; back handling.
Language	First-run and Settings language routes; medium layout on small/large screens; native ad loaded/failed; RTL if supported; immediate locale update.
Onboarding	Three slides, swipe, Skip, Next, Get Started; animations smooth; native ad does not cover controls; reduced-motion behavior.
Interstitial	Click 1–2 no request; click 3 one load; click 5 show only if ready; late load discarded; no-fill/error cooldown; next cycle begins only after impression and dismiss; no duplicate loads.
App Open	Minimize/resume; request only on qualifying resume; loading text; ready ad shown on same opportunity; back blocked only while overlay active; failed load recovers; no reload on dismiss; cooldown and frequency caps.
Paywall	No mic icon; 5–6 lines; compact plans; correct localized store price; Continue; legal links; cancel/restore/failure paths; no accidental double presentation.
Remote	Connected and disconnected states; supported/unsupported controls; D-pad, OK, Back, Home, volume, mute, playback; receiver-specific behavior; control latency and feedback.
Local-first	Airplane mode still allows local settings/history; no custom backend endpoints; data survives restart; clear data behavior; no accidental media upload.
Visual / performance	Light and dark themes; responsive layouts; 60 fps scrolling target; no clipped ad/remote controls; animation performance; memory leak checks on ad dispose.

Ad-specific instrumentation for QA

In debug builds only, expose a local diagnostics panel showing placement, request count, load state, impression count, last request timestamp, click counter, cooldown and last failure. This is local diagnostic data, not a production analytics backend.

11. Risks, assumptions and decisions to confirm

Topic	Risk / open point	Recommended decision
Two first-run paywalls	Requested sequence appears to present Paywall A and Paywall B during first launch.	Confirm both are intentional. Implement as two configurable slots so one can be disabled without changing routing.
“Entry” screen	Exact UI purpose is not fully described.	Treat as a minimal entry/continue gate. Define copy and CTA before design sign-off.
App Open overlay	Blocking back during loading can trap users if load hangs.	Add strict timeout and failure escape; block back only while the ad overlay is active, never indefinitely.
No backend + subscriptions	Client-only entitlement checks are less secure.	Use native store billing and document the limitation; do not add server validation without approval.
Remote compatibility	Not every TV supports all remote keys.	Capability-based controls and real-device testing per receiver.
Native ad in onboarding	Could damage the first-run experience or compress slides.	Reserve a stable native ad area; never overlay essential onboarding content.
Ad request timing	Late loads and repeated requests can cause unexpected show timing.	Use placement-specific state machines, one in-flight request and explicit expiry/cooldown rules.

12. Delivery phases

Phase	Deliverables	Exit gate
1. Product & UX	Confirm first-run/returning route, Entry screen, paywall copy/pricing, language and remote wireframes, ad placements.	Approved flow diagram and clickable UI prototype.
2. Core foundation	Flutter project, theme, router, local storage, AppFlowController, platform interfaces.	Cold start and routing pass on Android; no backend dependency.
3. Monetization	AdManager, native placements, click-3/load click-5/show interstitial, App Open resume lifecycle, billing/paywall.	Automated fake-ad tests plus real ad SDK test-device validation.
4. First-run UX	Language, onboarding slides and animations, native placements, persistence.	Fresh install and returning-user flows pass end-to-end.
5. Home & remote	Home dashboard, TV Remote tile, remote LCD-style layout and capability mapping.	Remote works on each supported receiver for the documented controls.
6. Casting core	Discovery, connection, screen mirroring, media casting and shared playback state.	Real Android phone + Chromecast + one DLNA TV validation.
7. Settings & polish	Pro card, language, theme, casting preferences, history, legal, feedback, latest version, help.	Settings persist; all links and controls verified.
8. Release & iOS prep	Android release QA, privacy/store declarations, crash checks; iOS platform audit and native spike.	Android release candidate accepted; iOS gap list documented.

13. Final implementation rules

• No custom backend, API server, Firebase project or cloud database in the baseline scope.

• Do not request or reload ads from impression/dismiss callbacks. Each placement has a documented trigger and lifecycle.

• Native ads load only on the screen where they are intended to appear.

• Interstitial load begins on click 3 and is eligible to show on click 5 only; late loads must not shift the show opportunity.

• App Open is tied to qualifying background-to-foreground transitions, with a recoverable loading overlay and safe back handling.

• Keep the paywall simple: no mic icon, five or six benefit lines, compact plans, one Continue CTA and legal/cancel text.

- Build a dedicated, capability-aware TV remote and expose it directly from Home.
- Preserve the existing soft theme while improving animation quality and screen hierarchy.
- Keep all shared logic Flutter/Dart-based and isolate Android/iOS native features behind stable interfaces.

Source alignment

This update builds on the uploaded “CastFlow – Screen Mirroring App: Complete Implementation Plan” (19 pages), retaining its local Wi-Fi casting concept, Flutter/Riverpod/go_router architecture, local persistence, receiver abstraction, Android MediaProjection and planned iOS ReplayKit path. New flow, ads, paywall, remote and settings requirements are based on the additional product instructions in this conversation.